

```
(ash (+ *small* *big*) -1))
GUESS-MY-NUMBER
```

Whenever we run a piece of code like this in the REPL, the resulting value of the entered expression will be printed. Every command in ANSI Common Lisp generates a return value. The *defun* command, for instance, simply returns the name of the newly created function. This is why we see the name of the function parroted back to us in the REPL after we call *defun*.

What does this function do? As discussed earlier, the computer's best guess in this game will be a number in between the two limits. To accomplish this, we choose the average of the two limits. However, if the average number ends up being a fraction, we'll want to use a near-average number, since we're guessing only whole numbers. We implement this in the *guess-my-number* function by first adding the numbers that represent the high and low limits, then using the arithmetic shift function, *ash*, to halve the sum of the limits and shorten the results. The built-in Lisp function *ash* looks at a number in binary form, and then shifts its binary bits to the left or right, dropping any bits lost in the process. For example, the number 11 written in binary is 1011. We can move the bits in this number to the left with *ash* by using 1 as the second argument:

```
> (ash 11 1)
22
```

We can move the bits to the right (and lop off the bit on the end) by passing in -1 as the second argument:

```
> (ash 11 -1)
5
```

Let's see what happens when we call our new function:

```
> (guess-my-number)
50
```

Since this is our first guess, the output we see when calling this function tells us that everything is working as planned: the program picked the number 50, right between 1 and 100.

Now, let's write our *smaller* and *bigger* functions. Like *guess-my-number*, these are global functions defined with *defun*:

```
> (defun smaller()
  (setf *big* (1- (guess-my-number))))
(guess-my-number))
SMALLER

> (defun bigger()
  (setf *small* (1+ (guess-my-number))))
```

```
(guess-my-number))
BIGGER
```

First, let's use *defun* to start the definition of a new global function, *smaller*. Next, use the *setf* function to change the value of our global variable **big**. Since we know the number must be smaller than the last guess, the biggest it can now be is 1 less than that guess. The code *(1- (guess-my-number))* calculates this: It first calls our *guess-my-number* function to get the most recent guess, and then it uses the function *1-*, which subtracts 1 from the result.

Finally, we want our smaller function to show us a new guess. We do this by putting a call to *guess-my-number* as the final line in the function body. This time, *guess-my-number* will use the updated value of **big**, causing it to calculate the next guess. The final value of our function will be returned automatically, causing our new guess (generated by *guess-my-number*) to be returned by the *smaller* function.

The *bigger* function works in exactly the same manner, except that it raises the **small** value instead. After all, if you call the *bigger* function, you are saying your number is *bigger* than the previous guess, so the smallest it can now be (which is what the **small** variable represents) is one more than the previous guess. The function *1+* simply adds 1 to the value returned by *guess-my-number*.

To complete our game, let's add a function *start-over*, to reset our global variables:

```
> (defun start-over()
  (defparameter *small* 1)
  (defparameter *big* 100)
  (guess-my-number))
```

As you can see, the *start-over* function resets the values of **small** and **big** and then calls *guess-my-number* again to return a new starting guess. Whenever you want to start a brand-new game with a different number, you can call this function to reset the game.

Figure 2 shows our game in action, with the number 74 as our guess.

In forthcoming articles, we'll go the A B C way, starting with the basic syntax and semantics of the language.

Power corrupts. Lisp is power. Study it hard. Be evil. And let's plan for world domination! 

By: Vivek Shangari

The author is a hard-core hacker, if ever there was one. He always thinks programs, and holds a piece of code in his head all the time. His favourite past time is to stop random strangers on the street and start talking about the benefits of open source over proprietary software, till they agree to switch sides, or threaten to jump off the nearest building. For those of you out there who cannot be deterred, you can reach him at vivek@codeCraft.in.